



2026
Software
Architecture
Specifications
Uni Textbook Marketplace

For Demo 2

Presented by:



NEXUSDEV

Software Architecture Specifications (SAS)

Project: Uni Textbook Marketplace
Team: NexusDev
Client: Agile Bridge
University: University of Pretoria - COS 301 Software Engineering
2026
Document version: 1.0 - Demo 2
Last updated: 29 July 2026

The Team

Team Member	Student number
1. Tiego Mokwena*	22496336
2. Gift Mohuba	23545527
3. Neo Bosoga	23591732
4. Josh Kretschmer	22883925

Note on scope: content that was under “Architectural Requirements” in the Demo 1 SRS (sections 7 and 8) has been moved into this document, as required for Demo 2. The SRS now only keeps requirements content going forward.

Contents

1	Introduction	4
2	Architectural Requirements	5
2.1	Architectural Patterns	5
2.2	Design Patterns	6
2.3	Constraints	6
2.4	Architectural Diagram	7
2.5	Mapping Quality Requirements to Architectural Decisions	7
3	Technology Requirements	9
4	API Contracts	11
4.1	Carried Over from Demo 1 (confirmed still in use)	11
4.2	Auth API Contracts	11
4.2.1	POST /auth/register	11
4.2.2	POST /auth/verify-otp	11
4.2.3	POST /auth/login	12
4.2.4	POST /auth/set-password	12
4.2.5	GET /auth/me	13
4.2.6	POST /auth/logout	13
4.2.7	GET /auth/universities	13
4.2.8	POST /auth/refresh	13
4.3	Listings API Contracts	13
4.3.1	POST /listings	13
4.3.2	GET /listings	14
4.3.3	GET /listings/mine	14
4.3.4	GET /listings/:id	14
4.4	Admin API Contracts	14
4.4.1	GET /listings/admin/all	14
4.4.2	PATCH /admin/listings/:id/approve	15
4.4.3	PATCH /admin/listings/:id/reject	15
4.5	Modules API Contract	15
4.5.1	POST /modules	15
4.5.2	GET /modules/faculties	15
4.6	New for Demo 2	15
4.7	Listing API Contract	15
4.7.1	PATCH /listings/editlist	15
4.8	Module API Contract	16

4.8.1	GET /modules (extended filters)	16
4.9	Saved Searches API Contract	16
4.9.1	POST /saved-searches/	16
4.9.2	GET /saved-searches/me	17
4.9.3	GET /saved-searches/id	17
4.9.4	DELETE /saved-searches/id	17
4.9.5	GET /saved-searches/me/email	17
4.9.6	GET /saved-searches/me/role	17
4.10	Wishlist API Contract	18
4.10.1	POST /wishlist/:listingId	18
4.10.2	DELETE /wishlist/:listingId	18
4.10.3	GET /wishlist/mine	18
4.11	Admin API Contract	18
4.11.1	GET /admin/audit-log	18
4.11.2	GET /admin/emails	19
4.12	Image API Contract	19
4.12.1	POST /images/upload	19
4.12.2	GET /images/list	19
4.12.3	DELETE /images/delete	19
4.13	Conversations API Contract	19
4.13.1	POST /conversations	20
4.13.2	POST /conversations/mine	20
4.13.3	POST /conversations/:id/messages	20
4.13.4	GET /conversations/:id/messages	20
5	Deployment	22
5.1	Overview	22
5.2	Live URLs	22
5.3	Deployment Diagram	23
5.4	CI/CD Pipeline Diagram	23
5.5	Deployment Requirements Status	25
5.6	Rollback Strategy	25
5.7	Issues Encountered and the Decisions That Came Out of Them	25

1. Introduction

The Uni Textbook Marketplace is a responsive web platform that lets verified university students buy, sell, and swap second-hand textbooks in a structured, module-aware way. The SRS covers what the system needs to do. This document covers how the system is built to actually do it: the architectural and design patterns we chose, our technology stack, our API contracts, and how the whole thing is deployed.

At a high level, the backend is a NestJS modular monolith, the frontend is Next.js, and Postgres (managed by Azure) handles persistence. Messaging is handled separately by Firebase Firestore. The system runs on Azure Container Apps across two environments, production and staging, with GitHub Actions handling both quality checks and deployment.

2. Architectural Requirements

2.1 Architectural Patterns

Modular Monolith (Core Backend)

The backend is a NestJS modular monolith. Instead of splitting everything into separate microservices from day one, all the core domain logic (Auth, Listings, Moderation, Modules, Saved Search, Wishlist, Audit Log) sits in one NestJS application, split into bounded modules. Each module has its own controller, service, and module file, and modules do not inject each other's services directly. We went with this because the team is small and the domain is well understood at this point, so splitting into microservices this early would just add operational overhead without much benefit. Since the modules are already cleanly separated, moving to microservices later, if it is ever needed, would not require a big rewrite.

External Microservice (Messaging)

In-app messaging (UC4) is handled by Firebase Firestore as a separate microservice, kept deliberately outside the core monolith. The frontend talks to Firestore directly through the client SDK, the NestJS backend does not sit in the middle of that traffic. This means if Firestore ever goes down, it will not take the rest of the platform with it. That is a graceful degradation boundary we built in on purpose.

Stateless API Layer

The NestJS API does not keep any session state on the server. Auth is handled through a JWT plus an HTTP-only cookie, with the cookie's settings changing depending on environment (`sameSite: 'none'` and `secure: true` once deployed, since frontend and backend are on different hostnames there, `sameSite: 'lax'` locally where that is not an issue). Because there is no server-side session state, any backend instance can serve any request, which is what makes horizontal scaling possible without needing sticky sessions.

Read-Heavy Performance Strategy

Browsing and searching listings happens a lot more than writing new ones, so the schema has indexes on the columns most used in filters and sorting: a composite index on (`module_id`, `price`), plus indexes on `condition`, `annotation_level`, `status`, `seller_id`, `reviewed_by`, and `created_at`. Books get indexes on `isbn`, (`author`, `title`), and `edition`. Audit log entries are indexed on (`entity_type`, `entity_id`) and `performed_at`.

Repository Pattern (Data Access)

Services never write raw SQL. All database access goes through TypeORM repositories, so the actual Postgres queries are abstracted away from the business logic. This is basically the object-persistence pattern covered in the course architecture slides (L11), where a

DB manager layer sits between business objects and the actual database access, so the business logic does not need to know or care what database is underneath.

2.2 Design Patterns

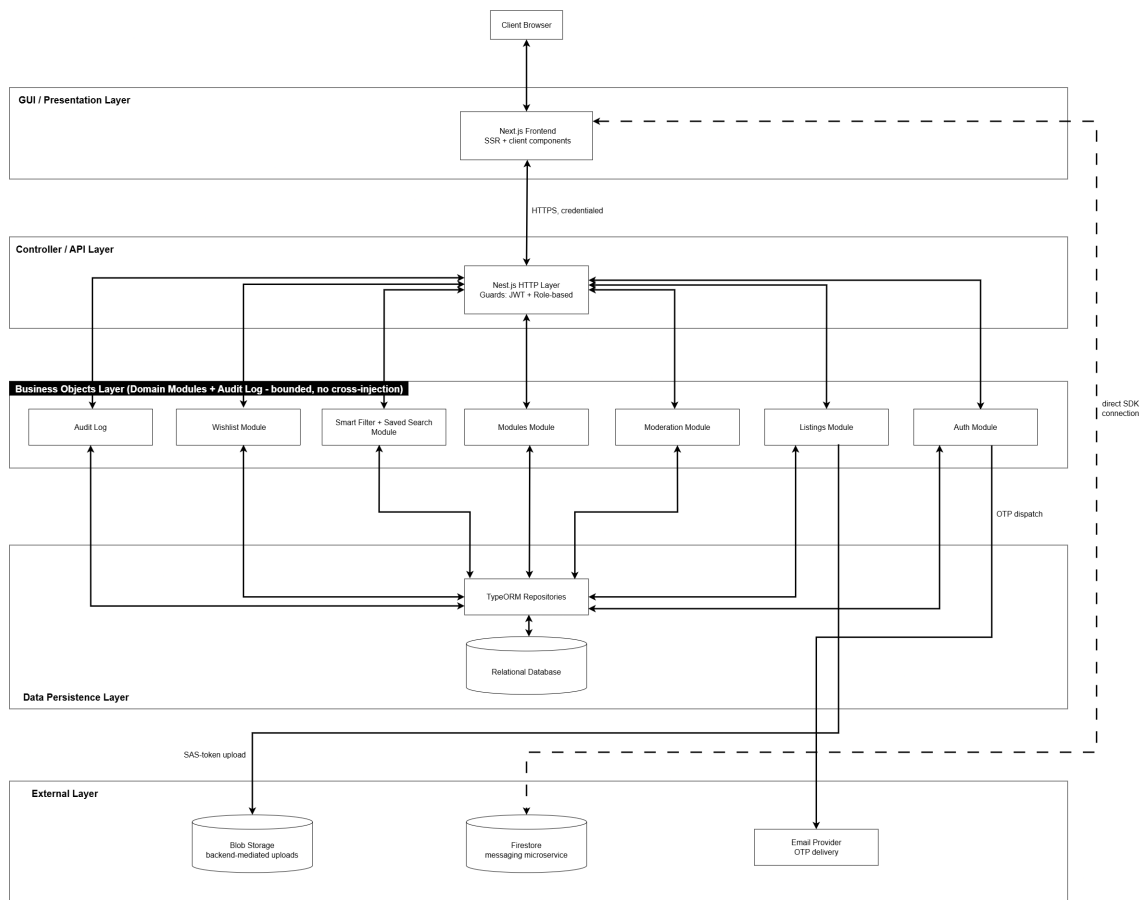
- **Decorator Pattern.** NestJS's guards (`@UseGuards(JwtAuthGuard, RolesGuard)`) add auth checks to a controller method without touching its actual logic. This keeps security concerns separate from what the endpoint is actually supposed to do.
- **Builder Pattern.** TypeORM's `createQueryBuilder()` is used for listing search. We start with a base query and add `andWhere()` clauses depending on which filters were actually sent in the request, instead of writing out a different hardcoded query for every possible filter combination.
- **Factory Pattern.** `ListingResponseFactory` is a simple factory that turns `Listing` entities into the objects the API actually returns, adding computed fields like a formatted price or a status label that are not stored in the database. This keeps the response shape consistent across every listing endpoint.

2.3 Constraints

Constraint	Detail
Hosting	Azure, under Agile Bridge's subscription (via client contact Kyle). We manage the deployment itself but not the underlying subscription or tenant.
Authentication	Custom OTP email verification, not a managed identity provider like Azure B2C.
Email allowlist	<code>@tuks.co.za</code> only, for now. This is configurable through backend environment variables.
No payment processing	The platform only facilitates meetup-based exchanges. No payment gateway, escrow, or financial transaction of any kind.
ISBN metadata	Manual entry only, no external ISBN lookup API.
No mobile app	Web only for now. A React Native app is a stretch goal, not something we are actively building.
Multi-university support	Only <code>tuks.co.za</code> is actually populated and supported right now, even though the data model itself allows for more than one university.

2.4 Architectural Diagram

This is a technology-neutral view of the system’s main components and how they talk to each other.



Note how messaging does not go through the NestJS gateway at all. That is the graceful degradation boundary from §2.1, shown visually.

2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision
Fast multi-filter listing search	Composite and single-column Postgres indexes on the columns used in listing search filters and sorting. Builder-pattern query construction avoids unnecessary full-table scans.
Core features stay available if messaging fails	Messaging is isolated as an external Firestore microservice, accessed directly by the frontend, so a Firestore outage cannot cascade into Auth, Listings, or Moderation.

Quality Requirement	Architectural Decision
Horizontal scalability	Stateless JWT-based auth means any backend instance can handle any request. Azure Container Apps can add replicas without needing sticky sessions.
Only verified students can create listings, only admins can moderate	OTP verification is required at registration. <code>JwtAuthGuard</code> and <code>RolesGuard</code> (Decorator pattern) sit on every protected route.
Modular, low-coupling codebase	NestJS bounded modules (Auth, Listings, Moderation, Modules, Saved Search, Wishlist, Audit Log), with no cross-module service injection.
WCAG AA accessibility	Enforced through automated Lighthouse audits. Currently sitting at roughly 95% AA compliance.
Measurable backend test coverage	<code>ci.yml</code> runs <code>npm run test:cov</code> against a real, ephemeral Postgres 15 service container on every push and PR to <code>main/develop</code> . Coverage is uploaded to Codecov.
Code quality and static analysis	SonarCloud runs on every push to <code>main/develop</code> and on every PR targeting either branch.
Reproducible, environment-parity deployment	Two fully separate Container App pairs for production and staging, each with its own database, both auto-deployed through SHA-tagged images (see §5).

3. Technology Requirements

Layer	Technology	Justification
Frontend	Next.js, React, TypeScript	Server-side rendering helps with perceived performance on the listing browse pages, which get hit the most. TypeScript keeps things type-safe across the codebase.
Frontend styling	Tailwind CSS	Utility-first styling means faster iteration, and the token-based theming lines up with the Agile Bridge brand style guide.
Backend	NestJS, TypeScript	The module structure maps directly onto our bounded-module architecture. Decorators cut down on repetitive guard and DTO code.
Database	Azure Database for PostgreSQL (Flexible Server)	Relational data fits our domain well, and it is fully ACID-compliant. Azure-native fits the hosting constraint.
ORM	TypeORM	Repository pattern for data access, plus a migration system for evolving the schema over time.
Authentication	Custom OTP plus JWT	Matches the client's OTP verification requirement. JWT is stateless, so there is no dependency on a managed identity provider.
File storage	Azure Blob Storage, backend-mediated with SAS tokens	Uploads go through the backend first rather than straight to Blob Storage, so storage credentials never touch the client.
Messaging	Firebase Firestore	Handles real-time chat as an external microservice. If it fails, the rest of the platform keeps working. The SDK also handles real-time subscriptions on its own, no backend proxying needed.

Layer	Technology	Justification
Hosting	Azure Container Apps	Container-native, and it has a built-in revision and rollback model that we have actually used in practice, which is part of why we picked it over App Service.
Container Registry	Azure Container Registry	Private image storage. <code>az acr build</code> lets us build remotely without needing a local Docker daemon.
CI (lint, test, coverage)	GitHub Actions, <code>ci.yml</code>	Runs on every push and PR to <code>main/develop</code> . Backend tests run against a real ephemeral Postgres service container, not a mock, so we get more confidence from the results.
CD (build, deploy)	GitHub Actions, <code>deploy.yml</code>	Branch-conditional, <code>main</code> deploys to production and <code>develop</code> deploys to staging. Images are tagged with the commit SHA, not <code>:latest</code> , so Container Apps always picks up a genuine change.
Code quality and static analysis	SonarCloud	Runs on every push to <code>main/develop</code> and on every PR targeting either branch.
Coverage reporting	Codecov	Separate <code>backend</code> and <code>frontend</code> flags per CI job.
Testing (backend)	Jest, plus a real ephemeral Postgres service container in CI	Gives integration-level confidence rather than just relying on a mocked database.
Testing (frontend)	Jest unit tests	Cypress end-to-end testing is planned but not implemented yet.
API documentation	Swagger (OpenAPI), generated from NestJS decorators	Auto-generated interactive API docs straight from the controller code.

4. API Contracts

All endpoints are prefixed with `/api`. All authenticated endpoints require the header `Authorization: Bearer <JWT>`. All request and response bodies use `Content-Type: application/json`.

4.1 Carried Over from Demo 1 (confirmed still in use)

4.2 Auth API Contracts

4.2.1 POST `/auth/register`

Description: Initiates student registration. Validates the email domain and triggers OTP dispatch.

Auth required: No

Request body:

```
{
  "fullName": "string (required)",
  "surname": "string (required)",
  "university": "string (required)",
  "email": "string (required, must be allowed domain)"
}
```

Success response (201 Created):

```
{
  "message": "Registration successful. Check your university email for verification code",
  "userId": "uuid"
}
```

Error responses:

- 400: Missing required field, or email domain not in allowlist
- 409: Email address is already registered

4.2.2 POST `/auth/verify-otp`

Description: Verifies the 6-digit OTP sent to the student's email.

Auth required: No

Request body:

```
{
  "email": "string (required)",
  "otp": "string (required, 6 digits)"
}
```

Success response (200 OK):

```
{
  "message": "Email verified successfully.",
  "verified": true
}
```

Error responses:

- 400: OTP is incorrect or has expired
- 404: No pending OTP found for this email

4.2.3 POST /auth/login

Description: Authenticates a verified student and returns a signed JWT.

Auth required: No

Request body:

```
{
  "email": "string (required)",
  "password": "string (required, min 8 characters)"
}
```

Success response (200 OK):

```
{
  "message" : "Login sucessful."
}
```

Error responses:

- 401: Invalid email or password
- 403: Account exists but email is not verified

4.2.4 POST /auth/set-password

Description: Sets the student's password after OTP verification is complete.

Auth required: No (uses `userId` from the registration step)

Request body:

```
{
  "userId": "uuid (required)",
  "password": "string (required, min 8 characters)",
  "confirmPassword": "string (required, must match password)"
}
```

Success response (200 OK):

```
{
  "message": "Password set successfully.",
  "accessToken": "string (JWT)"
}
```

4.2.5 GET /auth/me

Description: Returns the authenticated user's profile.

Auth required: Yes

4.2.6 POST /auth/logout

Description: Clears the session cookie.

Auth required: Yes

4.2.7 GET /auth/universities

Description: Lists supported universities, for the registration dropdown.

Auth required: No

4.2.8 POST /auth/refresh

Description: get a new access token using a refresh token, allowing users to stay logged in without typing their password again.

Auth required: No

4.3 Listings API Contracts

4.3.1 POST /listings

Description: Creates a new textbook listing. The listing is created with `status = PENDING`.

Auth required: Yes (Student role)

Request body:

```
{
  "isbn": "string (optional)",
  "title": "string (required)",
  "author": "string (optional)",
  "edition": "integer (required)",
  "condition": "new | good | fair | poor (required)",
  "annotationLevel": "none | light | heavy (required)",
  "bookId": "uuid (required)",
  "moduleId": "uuid (required)",
  "price": "number, positive decimal (required)",
  "hasNotes": "boolean (required)",
  "photoUrls": ["string (required >= 4)"],
  "description": "string (required)"
}
```

4.3.2 GET /listings

Description: Returns all APPROVED listings, with pagination and filters.

Auth required: Yes

Query parameters: moduleId, faculty, condition, minPrice, maxPrice, edition, page, limit

4.3.3 GET /listings/mine

Description: Returns all listings belonging to the authenticated student, including PENDING, APPROVED and REJECTED listings.

Auth required: Yes (Student role)

4.3.4 GET /listings/:id

Description: Returns the full detail object for a single listing.

Auth required: Yes

Path parameter: id, the UUID of the listing

4.4 Admin API Contracts

4.4.1 GET /listings/admin/all

Description: Returns all listings for Admin only.

Auth required: Yes (Admin role only)

4.4.2 PATCH /admin/listings/:id/approve

Description: Approves a pending listing. Sets status to APPROVED and writes an audit log entry.

Auth required: Yes (Admin role only)

4.4.3 PATCH /admin/listings/:id/reject

Description: Rejects a pending listing. Sets status to REJECTED, and writes an audit log entry also a reason is include for the reject.

Auth required: Yes (Admin role only)

Note: The audit log write was introduces for demo 2

4.5 Modules API Contract

4.5.1 POST /modules

Description: for creating a module associated with the textbook sold. Takes module details like name, code, semester and faculty id.

4.5.2 GET /modules/faculties

Description: Returns faculties name and id. Used when creating a module during listing creating

4.6 New for Demo 2

4.7 Listing API Contract

4.7.1 PATCH /listings/editlist

Module: Listings (UC5)

Description: Edits the details of an existing listing that the student owns. Only works for listings that are rejected or pending

Auth required: Yes (Student role, listing owner)

Request body: Only listing details can be edited, all is optional except listingid.

```
{
  "id" : "string (uuid)",
  "title" : "string",
  "condition" : 'new' | 'good' | 'fair' | 'poor',
  "annotation_level" : 'none' | 'light' | 'heavy',
  "price" : "number",
```



```
"has_notes" : "boolean",  
"description" : "string",  
}
```

4.8 Module API Contract

4.8.1 GET /modules (extended filters)

Module: Smart Filters (UC6)

Description: Extends the existing module search endpoint with additional smart filter parameters.

Description: Search as you type module code lookup. Returns modules matching the search query.

Auth required: Yes

Query parameters:

- **search** (string, required): partial module code or name to search
- **university** (string, optional): filter by university name (e.g. "UP")

4.9 Saved Searches API Contract

4.9.1 POST /saved-searches/

Module: Saved Search (UC6-ext)

Description: save a search as json **Request body:** all are optional

```
{  
  "filter_json" : {  
    "faculty" : "string",  
    "moduleCode" : "string",  
    "edition" : "string",  
    "priceMin" : "string | number",  
    "priceMax" : "string | number",  
    "condition" : "string",  
    "annotationLevel" : "string",  
    "search" : "string",  
    "book_title" : "string",  
    "author" : "string",  
    "isbn" : "string",  
    "university_id" : "string",  
    "faculty_id": "string"  
  }  
}
```

```
}
```

4.9.2 GET /saved-searches/me

Module: Saved Search (UC6-ext)

Description: Returns the saved searches belonging to the authenticated student. Route confirmed to exist in `saved_search.controller.ts`.

Auth required: Yes

Request body: uses query parameters.

- `id` (string)
- `email` (string)
- `role` (string)
- `firstname` (string)
- `lastname` (string)

4.9.3 GET /saved-searches/id

Module: Saved Search (UC6-ext)

Description: Returns a saved search by id **Auth required:** Yes

Request body: uses query parameters.

- `id` (string)

4.9.4 DELETE /saved-searches/id

Module: Saved Search (UC6-ext)

Description: Deletes a saved search by id **Auth required:** Yes

Request body: uses query parameters.

- `id` (string)

4.9.5 GET /saved-searches/me/email

Module: Saved Search

Description: Returns the authenticated student's saved search email.

4.9.6 GET /saved-searches/me/role

Module: Saved Search

Description: Returns role-specific saved search settings for the authenticated user.

4.10 Wishlist API Contract

4.10.1 POST /wishlist/:listingId

Module: Wishlist (UC7)

Description: Adds a listing to the authenticated student's wishlist.

Auth required: Yes (Student role)

Request body:

```
{
  "userid" : "string (uuid)",
  "listingid" : "string(uuid)",
}
```

4.10.2 DELETE /wishlist/:listingId

Module: Wishlist (UC7)

Description: Removes a listing from the authenticated student's wishlist.

Auth required: Yes (Student role)

Request body:

```
{
  "userid" : "string (uuid)",
  "listingid" : "string(uuid)",
}
```

4.10.3 GET /wishlist/mine

Module: Wishlist (UC7)

Description: Returns the authenticated student's wishlisted listings.

Auth required: Yes (Student role)

4.11 Admin API Contract

4.11.1 GET /admin/audit-log

Module: Audit Log (UC8)

Description: Returns audit log entries for admin review.

Auth required: Yes (Admin role only)

Query parameters: all are optional

- **performedBy** (string): filter by admin who performed the action
- **action** (string): approved a listing or rejected a listing
- **entityType** (string): Type action performed on, currently we have only listing
- **entityId** (string)
- **startDate** (string)
- **endDate** (string)

4.11.2 GET /admin/emails

Module: Audit Log (UC8)

Description: Return lists of admin emails, used for performing filters **Auth required:** Yes (Admin role only)

4.12 Image API Contract

4.12.1 POST /images/upload

Module: Blob upload

Description: Uploads a listing image to Azure Blob Storage via the backend.

Auth required: Yes (Student role)

4.12.2 GET /images/list

Module: Blob upload

Description: Lists uploaded images for a listing.

Auth required: Yes

4.12.3 DELETE /images/delete

Module: Blob upload

Description: Deletes an uploaded listing image from Blob Storage.

Auth required: Yes (Student role, listing owner)

4.13 Conversations API Contract

4.13.1 POST /conversations

Module: Messaging

Description: Create or retrieve a conversation for a listing

Auth required: Yes

Request body:

```
{
  "listingId": "uuid"
}
```

4.13.2 POST /conversations/mine

Module: Messaging

Description: Get all conversations for the logged in user

Auth required: Yes

4.13.3 POST /conversations/:id/messages

Module: Messaging

Description: Get all messages in a conversation

Auth required: Yes

Api param:

```
{
  name: 'id',
  description: 'Conversation ID',
}
```

4.13.4 GET /conversations/:id/messages

Module: Messaging

Description: Send a message in a conversation

Auth required: Yes

Api param:

```
{
  name: 'id',
  description: 'Conversation ID',
}
```

Request body:

```
{  
  "text" : "string"  
}
```

Note: the notification-core endpoints (event bus, notifications table, bell icon) are **not included above**. That work has been moved to Demo 3.

5. Deployment

5.1 Overview

The system is deployed as two separate services, a NestJS backend and a Next.js frontend, both packaged as Docker images and run on Azure Container Apps. A managed Azure Database for PostgreSQL Flexible Server handles persistence, and Azure Blob Storage handles listing image uploads through the backend, using SAS-token authentication.

There are two environments, production (served from `main`) and staging (served from `develop`), which is what satisfies the environment parity requirement. Both share one Container Apps Environment to keep costs down, but they are fully separated at the application and data level.

	Production	Staging
Trigger branch	<code>main</code>	<code>develop</code>
Backend app	<code>nexusdev-backend</code>	<code>nexusdev-backend-staging</code>
Frontend app	<code>nexusdev-frontend</code>	<code>nexusdev-frontend-staging</code>
Database	<code>nexustextbook</code>	<code>nexustextbook_staging</code>
Scaling	<code>min-replicas: 1,</code> always warm	<code>min-replicas: 0,</code> scales to zero when idle

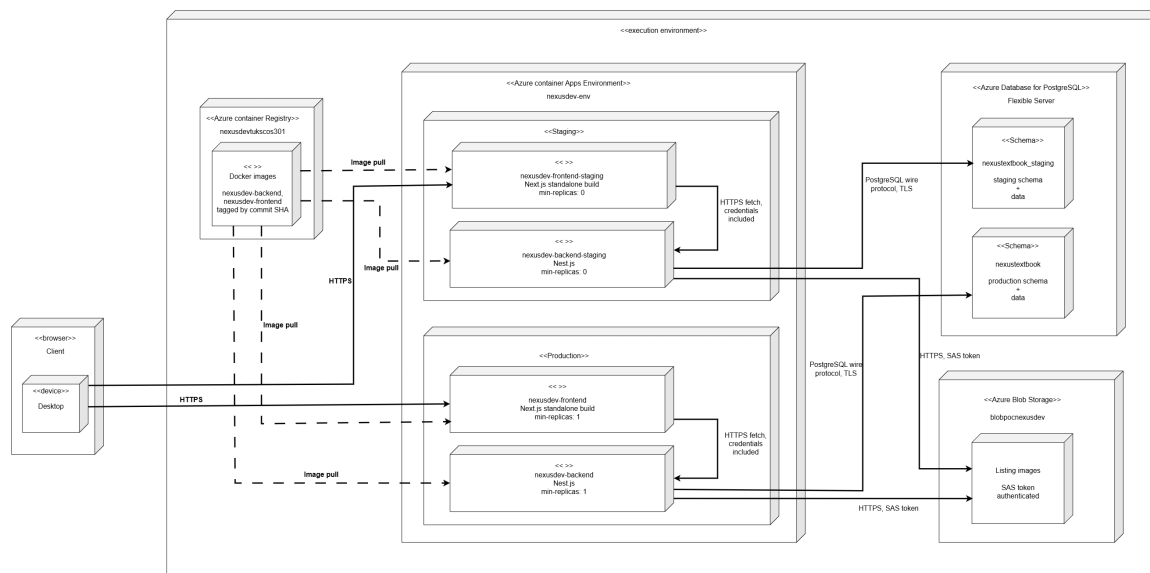
We deliberately chose one shared Container Apps Environment with separate databases, rather than two fully separate environments. A second full environment felt like unnecessary cost for a student capstone project, and the part that actually matters for correctness, data isolation, is already handled by giving each environment its own Postgres database on the same server. Staging runs at `min-replicas: 0`, which means the first request after some idle time takes a bit longer to respond (a cold start), but it costs close to nothing while nobody is using it. Production stays warm at `min-replicas: 1` since it has to respond immediately for real users at any time.

5.2 Live URLs

- **Production frontend:** <https://nexusdev-frontend.whitesand-df72b78b.southafricanorth.azurecontainerapps.io>
- **Production backend:** <https://nexusdev-backend.whitesand-df72b78b.southafricanorth.azurecontainerapps.io/api>

- **Staging frontend:** <https://nexusdev-frontend-staging.whitesand-df72b78b.southafricanorth.azurecontainerapps.io>
- **Staging backend:** <https://nexusdev-backend-staging.whitesand-df72b78b.southafricanorth.azurecontainerapps.io/api>

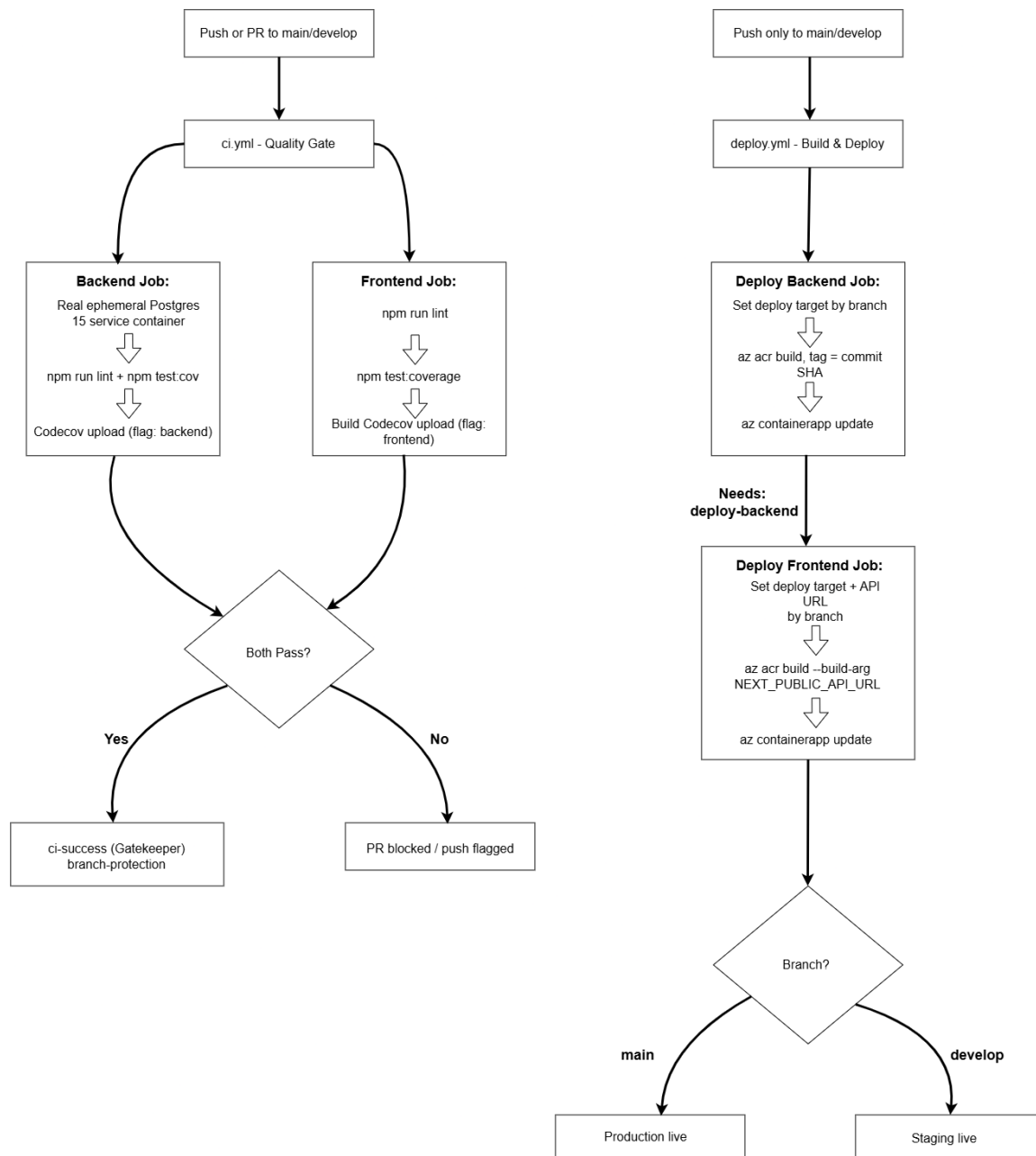
5.3 Deployment Diagram



This single diagram covers both environments, since they run on identical topology and only differ in which specific app and database instance they point at (labelled per node above). Region is South Africa North, resource group is **NexusDev**.

5.4 CI/CD Pipeline Diagram

There are two separate GitHub Actions workflows here, each doing a different job.



A few notes on how this actually works:

- `ci.yml` runs on both push and pull request to `main/develop`. This is the real quality gate. `deploy.yml` only runs on push, since an open PR should not trigger a deployment.
- Branch protection is configured on both `main` and `develop`. Every merge has to come in through a PR from a feature branch, needs at least one reviewer to approve, and the `ci-success` check has to pass. A failed CI run blocks the merge outright.
- Inside `deploy.yml`, `deploy-frontend` only runs after `deploy-backend` succeeds, since the frontend build needs the backend's live URL as a build-time argument.
- Every image is tagged with the full commit SHA, never `:latest`. This was actually a real bug we hit early on, Container Apps sometimes would not recognise that a

:latest image had actually changed, so we switched to SHA tags to make sure every deploy is unambiguous.

5.5 Deployment Requirements Status

Requirement	Status	Notes
Live, accessible system	Done	Both environments reachable over public HTTPS (§5.2)
Environment parity	Done	Production and staging each have their own Container Apps, their own database, and their own CI/CD trigger
Infrastructure as Code / Containerization	Done	Docker multi-stage builds, deployed through Azure Container Apps
Secrets management	Done	All credentials live as Container Apps secrets or GitHub Actions secrets, never committed to the repo
Rollback strategy	Done	See §5.6

5.6 Rollback Strategy

Azure Container Apps keeps previous revisions around. If a deployment goes wrong, we can shift traffic back to the last known good revision without rebuilding anything:

```
az containerapp revision list --name nexusdev-backend --resource-group NexusDev
  --output table
az containerapp ingress traffic set --name nexusdev-backend --resource-group
  NexusDev --revision-weight <previous-revision-name>=100
```

This is not just theoretical, we actually used this exact process during initial production deployment to fix a stale image causing CORS issues.

5.7 Issues Encountered and the Decisions That Came Out of Them

Issue	Root Cause	Decision Made
Stale <code>:latest</code> tag not recognised as changed by Container Apps	The tag string itself never changes, so the platform assumed nothing had	Every image is tagged with the commit SHA instead
Cross-origin cookie not staying logged in after login	<code>sameSite: 'lax'</code> blocks cross-origin fetch, and frontend and backend live on different hostnames once deployed	Cookie settings now change based on environment, <code>sameSite: 'none'</code> and <code>secure: true</code> when deployed, <code>'lax'</code> locally
Fresh database had no seed data, so registration always failed	The seed scripts had never actually been run against a genuinely empty database before	Confirmed the seed scripts work correctly against both a fresh staging database and a patched production database
Two migrations failed the same way on staging and, as it turned out, on production too	Migration class names did not match their required timestamp suffix, and a leftover <code>name</code> property on the class kept overriding the fix even after the class name was corrected	Fixed both migration files properly, and manually applied the corrected SQL to production directly since it had the exact same problem
The frontend image accidentally got deployed into the backend's Container App on staging	A copy-paste mistake in <code>deploy.yml</code> 's branch logic set the wrong app name in one place	Fixed the value, then confirmed with <code>az containerapp show</code> that each app was finally running the correct image
Staging was costing money even while nobody was using it	<code>min-replicas: 1</code> keeps a Container App running all the time no matter what	Set <code>min-replicas: 0</code> on both staging apps, accepting a short cold start on the first request after it has been idle

Uni Textbook Marketplace

In collaboration **with:**



2026 NexusDev - University of Pretoria - COS 301 Capstone Project
This document is version-controlled in the GitHub repository under `/docs/Software
Architecture Specifications.pdf`